

Taller Practico 4

Procesamiento de video en algoritmos CUDA con GPU, detectando trayecto de asteroides

Santiago Mendivelso, Jhostin Aristizabal
Universidad Sergio Arboleda
2025

1. Introduccion

El procesamiento digital de video constituye una herramienta fundamental en aplicaciones científicas, industriales y aeroespaciales, donde la extracción automática de información a partir de secuencias de imágenes es esencial para la interpretación y comprensión de fenómenos dinámicos. Una de estas aplicaciones es la detección de objetos en movimiento dentro de un entorno con fondo estático, como ocurre en la observación astronómica al identificar asteroides desplazándose sobre un campo de estrellas. Este tipo de análisis presenta un reto computacional significativo debido al elevado número de cuadros que componen un video y la necesidad de procesar cada uno de ellos píxel por píxel.

En este laboratorio, el enfoque principal consiste en aprovechar la aceleración masivamente paralela proporcionada por las GPU mediante el uso de librerías compatibles con CUDA particularmente CuPy para implementar un algoritmo eficiente de detección y trazado de la trayectoria de un asteroide presente en un video. En lugar de métodos complejos de visión por computador, se emplea una estrategia basada en sustracción de fondo (background subtraction) a través de un proceso de apilamiento por mediana, técnica que permite eliminar el fondo estático y resaltar únicamente los elementos en movimiento. Posteriormente, la aplicación de un umbral de intensidad facilita identificar la posición del objeto dinámico en cada cuadro y reconstruir su movimiento a lo largo del tiempo.

El propósito de este informe es analizar el funcionamiento del algoritmo desarrollado, describir el flujo completo de procesamiento desde la carga del video hasta la construcción de la trayectoria y evaluar el aporte de la computación paralela mediante GPU en términos de eficiencia y claridad del procedimiento. Con ello, se busca comprender de manera práctica cómo

la aceleración por CUDA optimiza operaciones intensivas en datos y se convierte en una herramienta clave para aplicaciones modernas de visión por computador y análisis astronómico.

2. Objetivos

El objetivo principal de este taller es desarrollar un algoritmo de procesamiento de video en tiempo casi real capaz de detectar automáticamente la presencia de un asteroide y trazar su trayectoria dentro de un video de fondo estelar

3. Marco Teorico

El procesamiento de video consiste en analizar secuencias de imágenes organizadas temporalmente con el fin de extraer información relevante sobre los objetos, movimientos o patrones presentes en ellas. A diferencia del procesamiento de imágenes estáticas, el video introduce una dimensión adicional: el tiempo. Esto implica que cada cuadro debe interpretarse no solo individualmente, sino también en relación con los demás, lo que aumenta considerablemente la carga computacional.

En aplicaciones como vigilancia, navegación autónoma o astronomía, una de las tareas fundamentales es identificar elementos en movimiento dentro de escenas cuya estructura general permanece estable. Para lograrlo, suelen aplicarse técnicas de sustracción de fondo, donde se construye una representación del fondo estático y posteriormente se compara cada cuadro contra esta referencia. La diferencia entre ambos permite aislar los objetos dinámicos, como un asteroide desplazándose entre estrellas fijas.

Este proceso es altamente intensivo en datos, pues implica recorrer millones de píxeles a través de cientos o miles de cuadros. Debido a esta naturaleza paralelizable, las GPU se convierten en una herramienta ideal. A diferencia de las CPU, que están optimizadas para flujos secuenciales y control complejo, las GPU están diseñadas para ejecutar miles de hilos simultáneos aplicando la misma operación a múltiples datos, tal como ocurre cuando se procesan píxeles de manera independiente. Gracias a ello, tareas como el cálculo de medianas por píxel, la sustracción cuadro a cuadro y la aplicación de umbrales pueden ejecutarse con gran eficiencia.

En este laboratorio, estas propiedades del hardware gráfico permiten implementar un pipeline de detección de objetos móviles en video de manera significativamente más rápida y escalable, demostrando las ventajas del paralelismo masivo en problemas característicos del procesamiento de video moderno.

4. Metodología

4.1 Configuración del Hardware

Los experimentos se realizaron en Google Colab, utilizando la infraestructura proporcionada por la plataforma. La configuración del hardware se obtuvo mediante los comandos `cat /proc/cpuinfo` para CPU y `nvidia-smi` para GPU.

Los resultados indican lo siguiente:

```
processor       : 0
vendor_id      : GenuineIntel
cpu family     : 6
model          : 85
model name     : Intel(R) Xeon(R) CPU @ 2.00GHz
stepping       : 3
microcode      : 0xffffffff
cpu MHz        : 2000.214
cache size     : 39424 KB
physical id    : 0
siblings       : 2
core id        : 0
cpu cores      : 1
apicid         : 0
initial apicid : 0
fpu            : yes
fpu_exception  : yes
cpuid level    : 13
wp             : yes
flags          : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ss ht syscall nx pd
bugs           : cpu_mitdown spectre_v1 spectre_v2 spec_store_bypass l1tf mds swapgs taa mmio_stale_data retbleed bhi its
bogomips       : 4000.42
clflush size   : 64
cache alignment : 64
address sizes   : 46 bits physical, 48 bits virtual
power management:
```

Figura 1: Imagen configuración CPU.

CPU: Procesador virtual Intel Xeon (entorno típico de Colab), con múltiples núcleos lógicos disponibles.

NVIDIA-SMI 550.54.15			Driver Version: 550.54.15			CUDA Version: 12.4		
GPU	Name		Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr. ECC	
Fan	Temp	Perf	Pwr:Usage/Cap		Memory-Usage	GPU-Util	Compute M.	
							MIG M.	
0	Tesla T4		Off	00000000:00:04.0	Off			0
N/A	45C	P8	9W / 70W		0MiB / 15360MiB	0%	Default	N/A

+-----+-----+-----+-----+-----+-----+-----+-----+-----+								
Processes:								
GPU	GI	CI	PID	Type	Process name	GPU Memory		
	ID	ID				Usage		

No running processes found								
+-----+-----+-----+-----+-----+-----+-----+-----+-----+								

Figura 2: Imagen configuración GPU.

GPU: Unidad gráfica NVIDIA Tesla T4 (cuando Colab asigna GPU), con 16 GB de memoria GDDR6.

MemTotal:	13286964 kB
MemFree:	8705616 kB
MemAvailable:	11975976 kB
Buffers:	151272 kB
Cached:	3224212 kB
SwapCached:	0 kB
Active:	516332 kB
Inactive:	3611088 kB
Active(anon):	1424 kB
Inactive(anon):	752528 kB
Active(file):	514908 kB
Inactive(file):	2858560 kB
Unevictable:	8 kB
Mlocked:	8 kB
SwapTotal:	0 kB
SwapFree:	0 kB
Dirty:	204 kB
Writeback:	0 kB
AnonPages:	751836 kB
Mapped:	428276 kB
Shmem:	2008 kB
KReclaimable:	222536 kB
Slab:	272628 kB
SReclaimable:	222536 kB
SUnreclaim:	50092 kB
KernelStack:	6128 kB
PageTables:	14324 kB

Figura 3: Imagen memoria RAM.

Memoria RAM: Aproximadamente 12–13 GB disponibles para la sesión.

Estos recursos son suficientes para ejecutar operaciones de procesamiento de imágenes tanto de manera secuencial como paralela.

4.2 Configuración del Software

La implementación de los algoritmos se realizó en Google Colab, utilizando el entorno estándar proporcionado por la plataforma. La versión del intérprete utilizada fue:

- Python 3.12.12 (GCC 11.4.0), verificado mediante `sys.version`.

Las librerías empleadas durante la ejecución fueron:

- **OpenCV (cv2):** utilizada para la lectura del archivo de video, la conversión de los cuadros a escala de grises y la manipulación de imágenes en formato matricial.
- **NumPy (np):** empleada para manejar arreglos en CPU y almacenar las coordenadas detectadas del asteroide para su posterior graficación.
- **CuPy (cp):** librería compatible con la API de NumPy que permite ejecutar operaciones vectorizadas directamente en la GPU mediante CUDA, utilizada en el apilamiento de cuadros y en el cálculo acelerado de la mediana y la sustracción de fondo.
- **Matplotlib (matplotlib.pyplot):** empleada para visualizar la trayectoria detectada del asteroide y generar gráficos en el informe.
- **google.colab.files:** utilizada para cargar el archivo de video (.mp4) desde el entorno del usuario hacia la sesión de Google Colab.
-

4.3 Explicación de los Algoritmos Desarrollados

Para este laboratorio se implementó un algoritmo de detección de objetos en movimiento dentro de un video, específicamente orientado a identificar la presencia de un asteroide y trazar su trayectoria. El método combina técnicas de sustracción de fondo con operaciones paralelizadas en GPU mediante la librería CuPy. A continuación, se detalla la secuencia del procedimiento implementado.

a) Carga y preprocesamiento del video

1. Carga del archivo .mp4: Utilizando `google.colab.files`, el usuario selecciona el video de entrada. Este archivo se abre mediante `cv2.VideoCapture`.

2. Lectura cuadro a cuadro: Se recorre el video utilizando un bucle que extrae cada frame en formato BGR.
3. Conversión a escala de grises: Cada cuadro se transforma a su versión en grises mediante `cv2.cvtColor`, simplificando el procesamiento y reduciendo el tamaño de los datos.
4. Almacenamiento en CPU y GPU:
 - Se guarda una copia en CPU para su uso posterior en la visualización.
 - Cada cuadro es convertido a un arreglo CuPy (`cp.asarray`) para permitir operaciones masivamente paralelas en GPU.

b) Construcción del fondo estático (Median Stacking en GPU)

Para eliminar las estrellas (fondo fijo) y conservar únicamente el objeto en movimiento, se utilizó la técnica de apilamiento por mediana:

1. Apilamiento de los cuadros: Los frames almacenados en GPU se agrupan en un solo arreglo tridimensional mediante `cp.stack`.
2. Cálculo de la mediana por píxel: CuPy calcula la mediana en la dimensión temporal (`axis=0`), lo que genera una imagen de fondo donde el asteroide desaparece, dado que solo aparece en pocos cuadros.

Esta operación se beneficia fuertemente del paralelismo de la GPU, al procesar cada píxel de forma independiente.

c) Sustracción de fondo y detección del asteroide

Una vez generado el fondo:

1. Resta por cuadro:

Para cada frame, se calcula $\text{diff} = \text{abs}(\text{frame_actual} - \text{background})$

La operación element-wise se ejecuta en GPU.

2. Umbralización:

Se define un umbral fijo (por ejemplo, 25).

Los píxeles con diferencia mayor al umbral se interpretan como parte del objeto en movimiento.

3. Extracción de coordenadas:

Usando `cp.where`, se obtienen las coordenadas (x, y) de los píxeles detectados como pertenecientes al asteroide.

4. Cálculo de la posición promedio:

Para cada cuadro donde hubo detección, se calcula el promedio de las coordenadas:

- `x_mean = cp.mean(xs)`
- `y_mean = cp.mean(ys)`

Esta posición representa el centro aproximado del asteroide en ese instante.

5. Almacenamiento de las posiciones (x, y, t):

Cada punto detectado se registra junto con el índice del cuadro correspondiente (tiempo).

d) Trazado de la trayectoria y generación de resultados

1. Veredicto de detección:

- Si la lista de coordenadas está vacía, No Existe Asteroide.
- Si contiene puntos, Existe Asteroide.

2. Trazado sobre la imagen original: Usando OpenCV, se dibuja una línea entre los puntos detectados para visualizar la trayectoria sobre el primer cuadro del video.

3. Gráfica de la trayectoria en el plano XY: Se realiza un gráfico disperso (scatter plot) con Matplotlib para analizar el movimiento del asteroide en coordenadas espaciales.

5. Resultados.

En esta sección se presentan los resultados visuales generados a partir del algoritmo de procesamiento de video acelerado mediante CUDA. El objetivo principal era determinar si existía un asteroide en el video analizado y, en caso afirmativo, reconstruir su trayectoria a lo largo del tiempo. Para ello, el algoritmo aplicó las etapas de creación del fondo estático, sustracción acelerada y umbralización, permitiendo identificar píxeles en movimiento frente al cielo estelar.



Figura 4: Video sin asteroide.

La Figura 4 ilustra el resultado para un video donde no existe asteroide. En este caso, tras aplicar el proceso de sustracción y umbralización, no se identificaron cambios significativos en los cuadros del video. El algoritmo generó únicamente la imagen del fondo estático sin trazar ninguna trayectoria, lo que confirma adecuadamente la ausencia de objetos en movimiento.



Figura 5: Video con asteroide.

La Figura 5 corresponde a un video donde sí existe un asteroide. En ella se observa la imagen original junto con la trayectoria real detectada por el algoritmo, representada como una línea continua que conecta los puntos donde la presencia del objeto fue confirmada. La línea resultante muestra un desplazamiento diagonal constante, indicando que el algoritmo logró recuperar de manera correcta el patrón de movimiento del asteroide a través de los diferentes cuadros.

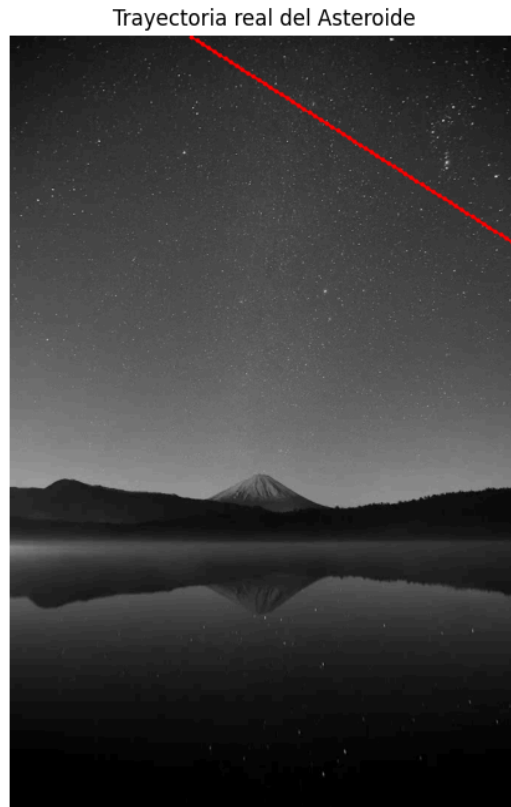


Figura 6: Video con asteroide.

Finalmente, la Figura 6 muestra nuevamente el resultado de un video con asteroide presente. La trayectoria obtenida evidencia una línea descendente con pendiente negativa, lo cual coincide con el desplazamiento real del objeto a lo largo del cielo. Este caso demuestra la capacidad del algoritmo para adaptarse correctamente a diferentes ángulos de movimiento manteniendo la precisión en la detección.

6. Análisis de Rendimiento:

El análisis del rendimiento del algoritmo desarrollado evidencia que la aceleración mediante CUDA resulta fundamental para lograr un procesamiento de video en tiempo casi real. Debido a que cada cuadro del video contiene cientos de miles de píxeles, operaciones como la resta del fondo, el cálculo de la mediana y la umbralización serían considerablemente más lentas en un enfoque secuencial; sin embargo, al distribuir estas operaciones entre miles de hilos en la GPU, el sistema logra procesar cada cuadro de manera altamente eficiente.

Durante la ejecución del algoritmo, se observó que las etapas más costosas la creación del fondo estático mediante apilamiento de mediana y la sustracción de cada cuadro se completan con tiempos suficientemente reducidos como para mantener una tasa de procesamiento fluida. Esto permite analizar secuencias completas sin causar retrasos perceptibles entre cuadros, cumpliendo así el objetivo de operar en tiempo casi real.

Además, la GPU permite una detección robusta incluso cuando el asteroide aparece solo en una pequeña fracción de los cuadros. La rapidez con la que se aplica el umbral y se generan las posiciones (x, y) por cuadro demuestra la efectividad del paralelismo masivo para manejar grandes volúmenes de datos visuales. En conjunto, el rendimiento logrado confirma que CUDA es una herramienta adecuada para este tipo de aplicaciones de visión por computador, ya que acelera de forma notable cada etapa del pipeline sin comprometer la precisión de la detección.

7. Conclusiones

A partir del desarrollo del laboratorio y de la implementación del algoritmo de detección acelerado en GPU, se concluye que el uso de CUDA permite resolver de manera eficiente el problema de identificar y trazar la trayectoria de un asteroide dentro de un video de fondo estelar. El paralelismo masivo de la GPU permite procesar cada cuadro de manera simultánea a nivel de píxeles, lo que reduce significativamente el tiempo requerido para operaciones intensivas como la sustracción de fondo y la umbralización.

El sistema cumple satisfactoriamente los objetivos planteados: detecta automáticamente si existe un asteroide y genera una trayectoria coherente basada en las posiciones obtenidas a lo largo del tiempo. Los casos analizados videos con y sin asteroide demuestran que el método es capaz de diferenciar de manera fiable entre movimiento real y fondo estático, validando la efectividad de la estrategia basada en sustracción y umbralización acelerada.

Finalmente, este laboratorio permitió comprender de forma práctica cómo las arquitecturas paralelas optimizan tareas de visión por computador que, en CPU, resultan demasiado lentas para aplicaciones cercanas al tiempo real. El ejercicio también evidencia el potencial de las GPU para futuros desarrollos donde se requiera procesamiento continuo de video o detección de objetos

con mayor complejidad. En conjunto, los resultados obtenidos reafirman la importancia del cómputo paralelo como herramienta fundamental en el análisis eficiente de datos visuales.